

EXPERIMENT 4 — Multiple Processes and Zombie Process

To create multiple child processes and synchronize their completion using `wait()` and `waitpid()`.

```
nano exp4_wait.c
```

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int i;
    pid_t pid;

    for (i = 0; i < 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            return 1;
        }

        if (pid == 0)
        {
            printf("Child %d: PID = %d, PPID = %d\n",
                i + 1, getpid(), getppid());

            sleep(i + 1);

            printf("Child %d finished: PID = %d\n",
                i + 1, getpid());

            exit(0);
        }
    }
}
```

```

    }
}

printf("\nParent PID = %d\n", getpid());
printf("Parent waiting for children...\n\n");

for (i = 0; i < 3; i++)
{
    wait(NULL);
}

printf("All children completed.\n");

return 0;
}

```

```

yaswanth6970@Yash6970:~$ nano exp4_wait.c
yaswanth6970@Yash6970:~$ gcc exp4_wait.c -o exp4_wait
yaswanth6970@Yash6970:~$ ./exp4_wait
Child 1: PID = 998, PPID = 997
Child 2: PID = 999, PPID = 997

Parent PID = 997
Parent waiting for children...

Child 3: PID = 1000, PPID = 997
Child 1 finished: PID = 998
Child 2 finished: PID = 999
Child 3 finished: PID = 1000
All children completed.

```

nano exp4_waitpid.c

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

```

```

int main()
{
    pid_t children[3];
    int i;

    for (i = 0; i < 3; i++)
    {

```

```
    children[i] = fork();

    if (children[i] < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (children[i] == 0)
    {
        printf("Child %d started. PID = %d\n",
            i + 1, getpid());

        sleep(3 - i);

        printf("Child %d finished. PID = %d\n",
            i + 1, getpid());

        exit(0);
    }
}

printf("\nParent waiting for specific children...\n");

for (i = 0; i < 3; i++)
{
    waitpid(children[i], NULL, 0);

    printf("Parent collected child PID = %d\n",
        children[i]);
}

printf("All children completed.\n");

return 0;
}
```

```

yaswanth6970@Yash6970:~$ nano exp4_waitpid.c
yaswanth6970@Yash6970:~$ gcc exp4_waitpid.c -o exp4_waitpid
yaswanth6970@Yash6970:~$ ./exp4_waitpid
Child 1 started. PID = 1041
Child 2 started. PID = 1042

Parent waiting for specific children...
Child 3 started. PID = 1043
Child 3 finished. PID = 1043
Child 2 finished. PID = 1042
Child 1 finished. PID = 1041
Parent collected child PID = 1041
Parent collected child PID = 1042
Parent collected child PID = 1043
All children completed.

```

wait()	waitpid()
Waits for any child process	Can wait for a specific child
Simpler	More control
Does not normally let you select a particular child	Allows selection using PID
Useful when any child can be collected	Useful when child order matters

nano zombie.c

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

```

```

int main()
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process exiting...\n");
        printf("Child PID = %d\n", getpid());
    }
}

```

```

        exit(0);
    }
    else
    {
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);

        printf("Parent sleeping for 30 seconds...\n");

        sleep(30);

        printf("Parent exiting...\n");
    }

    return 0;
}

```

```

yaswanth6970@Yash6970:~$ nano zombie.c
yaswanth6970@Yash6970:~$ gcc zombie.c -o zombie
yaswanth6970@Yash6970:~$ ./zombie
Parent PID = 1087
Child PID = 1088
Parent sleeping for 30 seconds...
Child process exiting...
Child PID = 1088
Parent exiting...

```

nano zombie_fixed.c

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

```

```

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
}

```

```

if (pid == 0)
{
    printf("Child process running...\n");
    printf("Child PID = %d\n", getpid());

    sleep(2);

    printf("Child exiting...\n");

    exit(0);
}
else
{
    printf("Parent PID = %d\n", getpid());
    printf("Child PID = %d\n", pid);

    printf("Parent waiting for child...\n");

    wait(NULL);

    printf("Child has been collected.\n");
    printf("No zombie remains.\n");
}

return 0;
}

```

```

yaswanth6970@Yash6970:~$ nano zombie_fixed.c
yaswanth6970@Yash6970:~$ gcc zombie_fixed.c -o zombie_fixed
yaswanth6970@Yash6970:~$ ./zombie_fixed
Parent PID = 1142
Child PID = 1143
Parent waiting for child...
Child process running...
Child PID = 1143
Child exiting...
Child has been collected.
No zombie remains.

```

controlling multiple child processes

OSSP Practical Experiment

Multiple children

One parent can create many children.

Parent

/ | \

/ | \

Child 1 Child 2 Child 3

What is wait()?

wait() tells the parent:

"Wait until a child finishes."

Parent

↓

wait()

↓

Child finishes

↓

Parent continues

What is waitpid()?

waitpid() lets the parent wait for a specific child.

For example:

waitpid(Child 2)

means:

"Wait for Child 2."

Easy difference:

wait()

↓

Wait for any child

|

waitpid()

↓

Wait for a specific child

What is a Zombie?

A zombie is a child process that has already finished, but the parent has not collected its result yet

Simple example:

Parent

|

Child

|

Child finishes

|

Zombie

The zombie is no longer doing any work. Linux just keeps a small amount of information about the finished child until the parent collects it.

You can see the zombie using

Bash

```
ps -o pid,ppid,state,cmd
```

If the state is:

Z

it means Zombie

How do we remove the zombie?

The parent uses:

C

```
wait()
```

or:

C

```
waitpid()
```

Then:

Child finishes

↓

Zombie

↓

Parent uses wait()



Zombie removed